Convolutional networks: local connectivity, weight sharing and pooling

Prof. Dr. Emre Onur Kahya · Department of Physics Engineering, Istanbul Technical University

A multilayer perceptron treats its input as an unordered list of numbers. Images, spectra, time series and lattice configurations are not unordered: neighbouring values are related, and the same pattern may appear anywhere. Convolutional networks build these two facts into the architecture. Each hidden unit looks only at a small patch (*local connectivity*), all units of a feature map use the same weights (*parameter sharing*), and *pooling* summarises small neighbourhoods. This note derives the discrete convolution and its gradients, assembles a complete network, shows how data augmentation supplies the invariances that the architecture lacks, follows the architectures from AlexNet to residual networks, and then works out the arithmetic of convolutions — output sizes, padding, strides, dilation, convolution as a sparse matrix, and the transposed convolutions used to produce images as outputs. Every operation comes with pseudocode and its cost.

1 Images and the three ideas

The goal of *object recognition* is to say which object an image contains. An image of height H and width W with C colour channels is an array of $C \times H \times W$ numbers: $C = 1$ for a grey-scale image, $C = 3$ for red, green and blue. Read as a vector, as in Note 1, Section 3, a 28×28 grey-scale digit gives 784 inputs and a modest 128×128 colour image $d = 3 \times 16\,384 = 49\,152$. Three properties distinguish such an input from a generic vector.

1. *It is high-dimensional.* A fully connected hidden layer with m units has $m(d + 1)$ parameters. For the $224 \times 224 \times 3$ images used on ImageNet ($d = 150\,528$), a single layer of 1000 hidden units already needs about 1.5×10^8 weights.
2. *It has a topology.* Pixels sit on a two-dimensional grid (three-dimensional for video or volumetric data, one-dimensional for audio or a spectrum), and nearby pixels are strongly correlated. A fully connected layer ignores this: permuting the pixels of every image in the same way would not change what it can learn.
3. *Its meaning is invariant under some transformations.* A cat shifted by ten pixels, slightly rotated or photographed in dimmer light is still a cat. Note 3, Section 4, listed such invariances as one reason why learning in very high dimension is possible at all, and Note 1, Section 3, showed with a three-pixel bar how badly raw pixel distances handle a one-pixel shift.

Convolutional networks (*convnets*, CNNs) [21, 22] exploit these properties with three ideas, which structure the first part of this note: *local connectivity* (Section 2), *parameter sharing* (Sections 2 and 3) and *pooling* (Section 4). The rest of the machinery — neurons, activation functions, layers, the softmax output and the cross-entropy loss — is that of the multilayer perceptron of Note 11, Section 4, and the gradients are computed by backpropagation, Note 12, Section 3. A convnet is an MLP with most of its weights forced to be zero and the remaining ones forced to be equal.

2 Local connectivity and parameter sharing

Local connectivity. Connect each hidden unit only to a small patch of the input, its *receptive field*, for instance a 3×3 or 5×5 square of pixels. The patch covers *all* channels: a unit looking at a 5×5 patch of a colour image has $5 \times 5 \times 3 = 75$ weights. Units at different positions look at different patches, so together they tile the image. The idea comes from neuroscience: Hubel and Wiesel found that neurons in the primary visual cortex of the cat respond to stimuli in a small region of the visual field, many of them to edges of a particular orientation [18], and Fukushima's neocognitron already stacked such local units [10].

Parameter sharing. Go one step further and give all units that look for the same kind of pattern *the same* weights, wherever their patch lies. Such a group of units forms a *feature map*: a two-dimensional grid of units, one per position, all computing the same function of their own patch. A layer has several feature maps (also called *channels*), each with its own weights, so that one map may respond to vertical edges, another to horizontal ones, a third to a colour contrast. Figure 1 contrasts the three kinds of layer in one dimension.

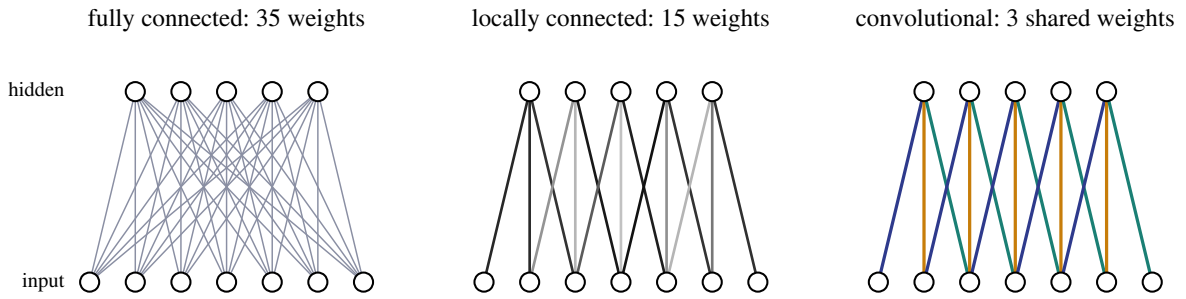


Figure 1. Three layers from seven inputs to five hidden units. Left: every unit sees every input. Centre: each unit sees three neighbouring inputs with its own weights (grey levels). Right: each unit sees three neighbouring inputs and all units use the same three weights (one colour per weight): a convolution.

> **Worked example — 1. Counting weights**
>
> Take a 32×32 grey-scale image and a hidden layer with one unit per position of a 30×30 grid (the positions where a 3×3 patch fits inside the image); ignore biases.
>
> - *Fully connected:* each of the 900 units sees all 1024 pixels: $900 \times 1024 = 921\,600$ weights.
> - *Locally connected:* each unit sees its own 3×3 patch with its own weights: $900 \times 9 = 8100$ weights.
> - *Shared:* all units use the same 3×3 weights: 9 weights, whatever the size of the image.
>
> With 64 feature maps instead of one, the shared layer has $64 \times 9 = 576$ weights and the fully connected layer $64 \times 921\,600 \approx 5.9 \times 10^7$. The computation shrinks less than the parameter count: the shared layer still evaluates 900×9 multiply–adds per map, as many as the locally connected one. Sharing saves *parameters* (and hence data and memory); locality saves both parameters and *operations*.

What sharing buys. Fewer parameters mean less data needed to fit them, and a feature detector learned at one position is immediately available at all others. Sharing also makes the layer *equivariant* to translations: shift the input, and every feature map shifts by the same amount, as Section 3.3 proves.

The two assumptions are a prior about the data (Note 4, Section 7.3): local statistics matter, and they are the same everywhere. When this is false — the input is a table of unrelated measurements, or the absolute position carries the meaning — convolutions are the wrong tool.

3 Discrete convolution

3.1 One dimension

Spatial indices start at 0 in this note, channel indices at 1. Let $x_0, \dots, x_{i-1}$ be a signal of length i and $w_0, \dots, w_{k-1}$ a *kernel* (or filter) of size k. The unit at position j of a shared, locally connected layer computes the *cross-correlation*

$$y_j = (x \star w)_j = \sum_{m=0}^{k-1} w_m \, x_{j+m}, \qquad j = 0, 1, \dots, i-k, \tag{1}$$

a weighted sum of the window $x_j, \dots, x_{j+k-1}$ that slides along the signal. The output has $o = i - k + 1$ entries, one for each position where the window fits entirely inside the signal. The *convolution* of signal processing and physics,

$$(x * w)_j = \sum_m w_m \, x_{j-m}, \tag{2}$$

runs through the kernel backwards. Restricted to the positions where the kernel fits, it equals the cross-correlation with the flipped kernel $\tilde{w}_m = w_{k-1-m}$, up to an index shift: substituting $m' = k - 1 - m$,

$$(x \star \tilde{w})_j = \sum_{m=0}^{k-1} w_{k-1-m} \, x_{j+m} = \sum_{m'=0}^{k-1} w_{m'} \, x_{j+k-1-m'} = (x * w)_{j+k-1},$$

so the valid outputs of the true convolution carry the indices $j = k-1, \dots, i-1$ instead of $0, \dots, i-k$. Since the weights are learned, the flip is immaterial: a network that needs w in one convention learns $\tilde{w}$ in the other. Deep-learning libraries implement (1) and call it convolution, and so shall we from now on. The flip matters only when hand-designed kernels are compared or when the mathematical properties of convolution are used: convolution is commutative and associative, cross-correlation is not.

> **Worked example — 2. A one-dimensional convolution by hand**
>
> Take $x = (2, 3, 5, 4, 1)$ and $w = (1, 0, -1)$. The window has three entries, so the output has $5 - 3 + 1 = 3$ entries:
>
> $$y_0 = 2 - 5 = -3, \qquad y_1 = 3 - 4 = -1, \qquad y_2 = 5 - 1 = 4.$$
>
> Each output is $y_j = x_j - x_{j+2} = -2\,(x_{j+2} - x_j)/2$, minus twice the central-difference estimate of the slope at $j+1$: the kernel measures the local slope of the signal, with a sign. The signal rises and then falls, and y goes from negative to positive accordingly. The true convolution (2) uses the flipped kernel $(-1, 0, 1)$ and gives $(3, 1, -4)$, the negative — for this antisymmetric kernel the flip only changes the sign. With the kernel $w = (1, 2, 0)$, cross-correlation gives $(8, 13, 13)$ and convolution $(11, 14, 9)$: in general the two differ by more than a sign.

3.2 Two dimensions and several channels

For an image $x_{u,v}$ and a $k \times k$ kernel $w_{a,b}$,

$$y_{u,v} = \sum_{a=0}^{k-1} \sum_{b=0}^{k-1} w_{a,b}\, x_{u+a,\, v+b}. \tag{3}$$

A layer of a convnet maps C_{in} input channels to C_{out} output channels. Its weights form a four-index array $W \in \mathbb{R}^{C_{\text{out}} \times C_{\text{in}} \times k \times k}$ and it has one bias b_c per output channel:

$$z_{c,u,v} = b_c + \sum_{c'=1}^{C_{\text{in}}} \sum_{a,b=0}^{k-1} W_{c,c',a,b}\, x_{c',\, u+a,\, v+b}, \qquad h_{c,u,v} = g(z_{c,u,v}), \tag{4}$$

with an element-wise activation g (ReLU in most modern networks, Note 11, Section 2; we write z, not a, for the pre-activation, because a, b index kernel offsets). Each output channel sums the cross-correlations of *all* input channels with its own slice $W_{c,c',:,:}$: a filter is a $C_{\text{in}} \times k \times k$ block, and the layer has

$$C_{\text{out}}\,(C_{\text{in}} k^2 + 1) \quad \text{parameters}, \tag{5}$$

independently of the image size. The first layer of AlexNet (Section 7), for example, has 96 filters of size $3 \times 11 \times 11$, that is $96 \times (363 + 1) = 34\,944$ parameters. The array z is the *pre-activation feature map*, h the feature map proper.

Worked example — 3. A two-dimensional convolution by hand

Take the 4×4 image and the 3×3 kernel

$$x = \begin{pmatrix} 1 & 2 & 0 & 1 \\ 0 & 1 & 3 & 1 \\ 2 & 1 & 0 & 0 \\ 1 & 0 & 1 & 2 \end{pmatrix}, \qquad w = \begin{pmatrix} 1 & 0 & -1 \\ 1 & 0 & -1 \\ 1 & 0 & -1 \end{pmatrix}.$$

The 3×3 window fits in 2×2 positions. With this kernel each output is the sum of the left column of the window minus the sum of its right column. Top left (rows 0–2, columns 0–2): $(1 + 0 + 2) - (0 + 3 + 0) = 0$. Top right (columns 1–3): $(2 + 1 + 1) - (1 + 1 + 0) = 2$. Bottom left (rows 1–3): $(0 + 2 + 1) - (3 + 0 + 1) = -1$. Bottom right: $(1 + 1 + 0) - (1 + 0 + 2) = -1$. Hence

$$y = x \star w = \begin{pmatrix} 0 & 2 \\ -1 & -1 \end{pmatrix}, \qquad \text{ReLU}(y + 1) = \begin{pmatrix} 1 & 3 \\ 0 & 0 \end{pmatrix}$$

with a bias of $+1$. Surrounding the image by a border of zeros (*zero padding*, Section 8) keeps the output at 4×4; moving the window two pixels at a time (*stride* 2) keeps only every second output. Both are used in Question B2.

Two input channels. With $C_{\text{in}} = 2$, one output channel sums the cross-correlations of both input channels with their own slices of its filter, as in (4). Take the 3×3 channels and 2×2 slices

$$x_1 = \begin{pmatrix} 1 & 2 & 0 \\ 0 & 1 & 3 \\ 2 & 1 & 0 \end{pmatrix}, \quad x_2 = \begin{pmatrix} 1 & 0 & 1 \\ 0 & 2 & 0 \\ 1 & 0 & 1 \end{pmatrix}, \quad W_{1,1} = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}, \quad W_{1,2} = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}.$$

$W_{1,1}$ adds the diagonal of each window of x_1, $W_{1,2}$ the anti-diagonal of each window of x_2; with bias $b_1 = 1$,

$$z_1 = \begin{pmatrix} 2 & 5 \\ 1 & 1 \end{pmatrix} + \begin{pmatrix} 0 & 3 \\ 3 & 0 \end{pmatrix} + 1 = \begin{pmatrix} 3 & 9 \\ 5 & 2 \end{pmatrix}.$$

A second output channel would repeat the computation with its own two slices.

3.3 Equivariance, and a convolution as a feature detector

Let $(T_s x)_j = x_{j-s}$ shift a signal by s positions (for an infinite signal, or away from the borders). Then

$$\big((T_s x) \star w\big)_j = \sum_m w_m \, x_{j+m-s} = (x \star w)_{j-s} = \big(T_s(x \star w)\big)_j \ :$$

shifting the input and then convolving gives the same as convolving and then shifting. The convolution *commutes* with translations; it is *translation equivariant*. The element-wise activation also commutes with shifts, so a whole convolutional layer with stride 1, and a stack of such layers, is equivariant (up to border effects); a layer with stride s is equivariant only to shifts by multiples of s (compare Worked example 5). A function f is *invariant* if $f(T_s x) = f(x)$ instead; equivariant maps keep track of *where* a pattern is, invariant ones forget it. Classification needs invariance at the end; Section 4 shows how pooling turns equivariance into approximate invariance.

> **Worked example — 4. An edge detector moves with the edge**
>
> A 5×5 image is dark (0) in its two left columns and bright (2) elsewhere; every row is $(0, 0, 2, 2, 2)$. The kernel with three identical rows $(-1, 0, 1)$ computes, at each window, (right column) − (left column), summed over three rows. Each row of the 3×3 output is $(3\cdot 2,\ 3\cdot 2,\ 0) = (6, 6, 0)$: large where the window straddles the dark-to-bright step, zero where it lies in the uniform bright region. Shift the edge one column to the right, so that every row reads $(0, 0, 0, 2, 2)$: every row of the output becomes $(0, 6, 6)$, the old response shifted by one column. The detector does not need to know where the edge is; the position is encoded in *which* output units respond.

Figure 2 applies two hand-made 3×3 edge filters, each followed by a ReLU, to a synthetic image. The first map lights up along dark-to-bright transitions from left to right, the second along transitions from top to bottom. Shifting the input by $(3, 4)$ pixels shifts the map by exactly $(3, 4)$ away from the borders, as the derivation above requires. A trained convnet finds such filters by itself (Section 7).

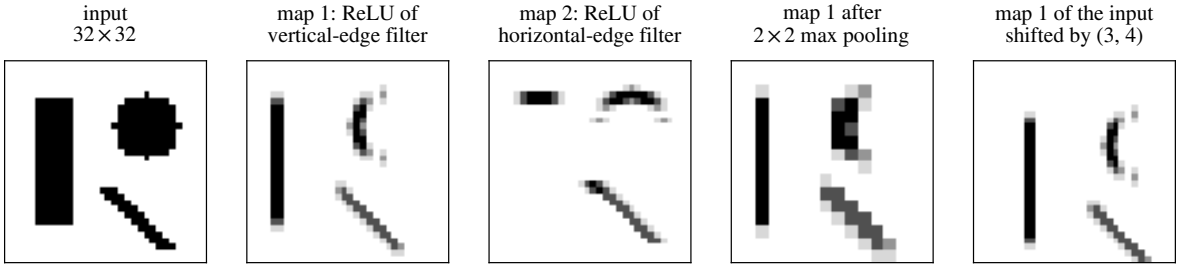


Figure 2. Feature maps of a 32 × 32 synthetic image (zero padding, so maps keep the image size). Map 1 uses the kernel with rows $(-1, 0, 1)$, $(-2, 0, 2)$, $(-1, 0, 1)$; map 2 its transpose. Fourth panel: map 1 after 2 × 2 max pooling, 16 × 16. Last panel: map 1 of the image shifted by 3 rows and 4 columns; in the interior it equals the second panel shifted by the same amounts.

> **Running time — convolutional layer, forward**
>
> *Time*: $O(C_{\text{out}} H' W' \, C_{\text{in}} k^2)$ multiply–adds per example: one dot product of length $C_{\text{in}} k^2$ per output unit. A 3×3 layer from 64 to 64 channels on a 56×56 map needs $56^2 \cdot 64 \cdot 64 \cdot 9 \approx 1.2 \times 10^8$ multiply–adds; a fully connected layer between the same two arrays would need $(56^2 \cdot 64)^2 \approx 4 \times 10^{10}$. *Memory*: $C_{\text{out}}(C_{\text{in}} k^2 + 1)$ parameters (36 928 in the example), plus the $C_{\text{out}} H' W'$ activations, which must be kept for the backward pass — in deep convnets the activations, not the weights, dominate memory during training. *Prediction*: the same as the forward pass. Libraries do not run these loops literally; Section 8 shows the matrix-product form they use.

Algorithm 1 Convolutional layer, forward pass (direct form)

Require: input $x \in \mathbb{R}^{C_{\text{in}} \times H \times W}$; weights $W \in \mathbb{R}^{C_{\text{out}} \times C_{\text{in}} \times k \times k}$; biases $\mathbf{b} \in \mathbb{R}^{C_{\text{out}}}$; stride s; padding p; activation g

function ConvForward($x; W, \mathbf{b}, s, p$)
  $\bar{x} \leftarrow x$ surrounded by p zeros on each side of each spatial axis ▷ Section 8
  $H' \leftarrow \lfloor (H + 2p - k)/s \rfloor + 1$; $W' \leftarrow \lfloor (W + 2p - k)/s \rfloor + 1$
  for $c = 1, \dots, C_{\text{out}}$; $u = 0, \dots, H' - 1$; $v = 0, \dots, W' - 1$ **do**
    $z_{c,u,v} \leftarrow b_c + \sum_{c'=1}^{C_{\text{in}}} \sum_{a,b=0}^{k-1} W_{c,c',a,b}\, \bar{x}_{c',\, su+a,\, sv+b}$ ▷ $C_{\text{in}} k^2$ multiply–adds
  end for
  $h \leftarrow g(z)$ element-wise
  return h (and keep $\bar{x}$, z for the backward pass)
end function

In physics — shift symmetry, Green's functions and Fourier modes

Physicists know convolutions as the response of linear, time-invariant systems: the output of a damped oscillator or an RC circuit is the input convolved with the impulse response (the Green's function), and the field of a charge distribution is the density convolved with the Coulomb kernel. This is no coincidence: any linear map on signals that commutes with all translations is a convolution (Question C1). Translation symmetry therefore fixes the *form* of a linear layer; learning only chooses the kernel. Like a translation-invariant Hamiltonian, such a map is diagonal in the Fourier basis, which is the convolution theorem: $\widehat{x * w} = \hat{x}\,\hat{w}$. The same reasoning extends to other symmetry groups. Group-equivariant CNNs share weights across rotations and reflections as well as translations [5], and networks for molecules and crystals are built to respect rotations and permutations of identical atoms — symmetry, not data, removes those degrees of freedom.

4 Pooling

The third idea is to summarise small neighbourhoods of each feature map. *Max pooling* over $m \times m$ blocks with stride m (non-overlapping blocks) computes, separately for each channel c,

$$y_{c,u,v} = \max_{0 \le a,b < m} h_{c,\, mu+a,\, mv+b}, \tag{6}$$

and *average pooling* replaces the maximum by the mean $\frac{1}{m^2} \sum_{a,b} h_{c,\, mu+a,\, mv+b}$. Pooling has no parameters, acts on each channel separately, and with $m = 2$ divides the height and the width by two, the number of units by four. Overlapping windows (window 3, stride 2, as in AlexNet) are also used.

Pooling does two things. It *subsamples*: later layers see a coarser grid and cost less. And it buys *local invariance*: the maximum of a block does not change when its largest entry moves within the block.

Worked example — 5. Max pooling and small shifts

Pool the 4×4 feature map h in 2×2 blocks:

$$h = \begin{pmatrix} 6 & 0 & 2 & 0 \\ 1 & 0 & 0 & 1 \\ 0 & 0 & 4 & 0 \\ 3 & 0 & 1 & 0 \end{pmatrix} \longmapsto \begin{pmatrix} 6 & 2 \\ 3 & 4 \end{pmatrix} \qquad \text{(average pooling: } \begin{pmatrix} 1.75 & 0.75 \\ 0.75 & 1.25 \end{pmatrix}\text{)}.$$

Shift h to the right by one or by two columns, filling with zeros, and pool again:

$$\text{shift by 1: } \begin{pmatrix} 6 & 2 \\ 3 & 4 \end{pmatrix}, \qquad \text{shift by 2: } \begin{pmatrix} 0 & 6 \\ 0 & 3 \end{pmatrix}.$$

After a shift by one, every maximum moves from the left column of its block to the right column of the same block, and the pooled map is unchanged: invariant. After a shift by two the maxima cross into the next block, and the result is the old pooled map shifted by one cell (its right column has left the image). A shift by a multiple of the pooling stride is reproduced, on the coarser grid, as a shift; smaller shifts are absorbed only when they do not carry a maximum across a block boundary. Pooling gives approximate, local invariance, not an exact one.

Gradients through pooling. Backpropagation (Note 12, Section 3) needs $\partial y/\partial h$. For max pooling, $y_{c,u,v}$ equals one entry of its block and does not depend on the others (where the maximum is unique), so the incoming gradient is routed entirely to the position of the maximum, the *argmax*, and the other entries of the block receive zero. For average pooling every entry of the block receives $1/m^2$ of it.

Algorithm 2 Max pooling ($m \times m$ blocks, stride m): forward and backward

Require: feature maps $h \in \mathbb{R}^{C\times H\times W}$ with H, W divisible by m

function MaxPoolForward(h; m)
  for $c = 1, \dots, C$; $u = 0, \dots, H/m - 1$; $v = 0, \dots, W/m - 1$ **do**
    $(a^*, b^*) \leftarrow \arg\max_{0\le a,b<m} h_{c,\,mu+a,\,mv+b}$; $y_{c,u,v} \leftarrow h_{c,\,mu+a^*,\,mv+b^*}$
    store (a^*, b^*) as $\arg_{c,u,v}$ ▷ needed by the backward pass
  end for
  return y, arg
end function
function MaxPoolBackward($\partial\ell/\partial y$, arg; m)
  $\bar{h} \leftarrow$ zero array of the shape of h
  for each (c, u, v), with $(a^*, b^*) = \arg_{c,u,v}$ **do**
    $\bar{h}_{c,\,mu+a^*,\,mv+b^*} \leftarrow \partial\ell/\partial y_{c,u,v}$ ▷ gradient goes to the winner only
  end for
  return $\bar{h} = \partial\ell/\partial h$
end function
AvgPoolBackward: $\bar{h}_{c,\,mu+a,\,mv+b} \leftarrow \frac{1}{m^2}\,\partial\ell/\partial y_{c,u,v}$ for all $0 \le a, b < m$

Running time — pooling

Time: $O(CHW)$ per example for both passes, since every input entry is compared (or added) once; this is small next to the convolutions. *Memory*: no parameters; the output has CHW/m^2 entries, and the forward pass stores one argmax index per output for training. Pooling divides the cost of every later layer by m^2. Global average pooling (Section 8) is the case $m = H = W$, at cost $O(CHW)$.

In physics — pooling and coarse-graining

Kadanoff's block-spin construction replaces each 3×3 block of Ising spins by one spin, for instance by majority rule, and asks how the couplings change under this coarse-graining; iterating it is a real-space renormalisation-group transformation. A stack of convolutions and pooling layers is reminiscent of this: local operations followed by subsampling, with fine detail discarded and large-scale structure kept. Mehta and Schwab made the analogy precise for one

particular model class, mapping variational renormalisation onto stacked restricted Boltzmann machines [26]. It should be read as an analogy, not an identity: the renormalisation group keeps what determines long-distance physics, while a classifier keeps whatever predicts its label. Note also that pooling, like block-spin averaging, cannot be undone — a fact that matters when an image must be produced as output (Section 10).

5 A complete convolutional network

A convolutional network alternates convolutional layers (with their activation) and pooling layers. The number of channels usually grows as the spatial size shrinks, so that each stage trades resolution for more kinds of features. At the end the remaining feature maps are flattened into a vector, or averaged over positions (Section 8), and fed to one or more fully connected layers and a softmax output exactly as in an MLP (Note 11, Sections 4 and 5). This is the design of LeNet [22], which read handwritten digits on bank cheques in the 1990s, and in outline of every convnet since.

Worked example — 6. Shapes and parameters of a LeNet-style network

Input: a $1\times28\times28$ grey-scale digit; ten classes. Convolutions have stride 1 and ReLU activations, pooling is 2×2 max pooling. Shapes are written channels × height × width; "multiply–adds" counts the forward pass for one image.

layer	operation	output shape	parameters	multiply–adds
conv 1	6 filters $1\times5\times5$, $p=2$	$6\times28\times28$	156	117 600
pool 1	2×2 max	$6\times14\times14$	0	—
conv 2	16 filters $6\times5\times5$, $p=0$	$16\times10\times10$	2 416	240 000
pool 2	2×2 max	$16\times5\times5=400$	0	—
fc 1	$400\to120$	120	48 120	48 000
fc 2	$120\to84$	84	10 164	10 080
fc 3 + softmax	$84\to10$	10	850	840
total			61 706	416 520

By (5), conv 1 has $6(25+1)=156$ parameters and conv 2 has $16(150+1)=2\,416$. Two features of convnets are visible. The convolutions hold 4% of the parameters but do 86% of the computation (each weight is reused at hundreds of positions), while the first fully connected layer holds most of the parameters. And a fully connected layer producing the same $6\times28\times28$ output as conv 1 would need $784\times4704+4704=3\,692\,640$ parameters instead of 156.

The receptive field grows with depth. A unit of conv 1 sees a 5×5 patch of the image. A unit of pool 1 sees 6×6 pixels (two neighbouring conv 1 windows), and one step on the pool 1 grid is two pixels. A unit of conv 2 sees 5 pool 1 positions, so $6+4\times2=14$ pixels in each direction, and a unit of pool 2 sees 16. In general, if layer l has kernel size k_l and stride s_l (pooling counts as a layer with $k=s=m$), the receptive field in input pixels obeys

$$r_l = r_{l-1} + (k_l - 1)\prod_{l'<l} s_{l'}, \qquad r_0 = 1, \tag{7}$$

because one step on the grid of layer $l-1$ is $\prod_{l'<l} s_{l'}$ input pixels. Without strides or pooling the receptive field grows only linearly, by $k-1$ per layer: ten 3×3 layers see 21×21 pixels. Strides and pooling make it grow geometrically, and dilation (Section 9) does so as well. Early layers thus detect small, local patterns and deep layers combine them over large regions — a hierarchy of features.

5.1 Backpropagation through a convolution

Training uses mini-batch stochastic gradients of the cross-entropy, exactly as for an MLP (Note 12, Section 5, gives the loop, Note 7, Sections 5 and 6, the optimisers). Only two new local derivatives are needed: through pooling (Section 4) and through a convolution. Consider the one-dimensional layer $y_j = b + \sum_m w_m x_{j+m}$, $j = 0, \dots, o-1$, and suppose backpropagation has delivered $\delta_j = \partial \ell / \partial y_j$ for every output.

Gradient with respect to the weights. The weight w_m appears in every output, multiplied by x_{j+m}, so by the chain rule

$$\frac{\partial \ell}{\partial w_m} = \sum_{j=0}^{o-1} \frac{\partial \ell}{\partial y_j} \frac{\partial y_j}{\partial w_m} = \sum_j \delta_j \, x_{j+m} = (x \star \boldsymbol{\delta})_m, \qquad \frac{\partial \ell}{\partial b} = \sum_j \delta_j. \tag{8}$$

The gradient of a shared weight is the *sum* of the gradients it would receive at each position: the cross-correlation of the input with the output gradient.

Gradient with respect to the input. The input x_q enters the outputs y_j with $j + m = q$, that is with weight w_{q-j}, for those j with $0 \le q - j \le k - 1$:

$$\frac{\partial \ell}{\partial x_q} = \sum_j \delta_j \, w_{q-j} = (\boldsymbol{\delta} * w)_q, \tag{9}$$

a *full* convolution (true convolution, kernel flipped) of the output gradient with the kernel, defined for all $q = 0, \dots, i-1$ by treating δ_j as zero outside $0 \le j < o$. Equivalently: pad $\boldsymbol{\delta}$ with $k-1$ zeros on each side and cross-correlate it with the flipped kernel. In two dimensions with channels the same steps give

$$\frac{\partial \ell}{\partial W_{c,c',a,b}} = \sum_{u,v} \Delta_{c,u,v} \, x_{c',\,u+a,\,v+b}, \qquad \frac{\partial \ell}{\partial x_{c',q,r}} = \sum_c \sum_{a,b} \Delta_{c,\,q-a,\,r-b} \, W_{c,c',a,b},$$

with $\Delta = \partial \ell / \partial z$ the gradient of the pre-activations, obtained from $\partial \ell / \partial h$ by multiplying with $g'(z)$ element-wise. With a mini-batch, the weight gradients are summed (or averaged) over the examples.

Worked example — 7. Backward pass through a one-dimensional convolution

Take $x = (1, 2, -1, 3)$, $w = (2, -1)$, $b = 0$, targets $t = (1, 4, -4)$ and the squared loss $\ell = \frac{1}{2}\sum_j (y_j - t_j)^2$. *Forward:* $y_0 = 2 \cdot 1 - 2 = 0$, $y_1 = 2 \cdot 2 + 1 = 5$, $y_2 = 2 \cdot (-1) - 3 = -5$; the residuals are $\boldsymbol{\delta} = y - t = (-1, 1, -1)$ and $\ell = 1.5$. *Weights:* by (8),

$$\frac{\partial \ell}{\partial w_0} = \delta_0 x_0 + \delta_1 x_1 + \delta_2 x_2 = -1 + 2 + 1 = 2, \qquad \frac{\partial \ell}{\partial w_1} = \delta_0 x_1 + \delta_1 x_2 + \delta_2 x_3 = -2 - 1 - 3 = -6,$$

and $\partial \ell / \partial b = -1$. *Input:* by (9), each x_q collects $\delta_j w_{q-j}$ from the outputs that saw it:

$$\frac{\partial \ell}{\partial x_0} = \delta_0 w_0 = -2, \quad \frac{\partial \ell}{\partial x_1} = \delta_1 w_0 + \delta_0 w_1 = 2 + 1 = 3, \quad \frac{\partial \ell}{\partial x_2} = \delta_2 w_0 + \delta_1 w_1 = -3, \quad \frac{\partial \ell}{\partial x_3} = \delta_2 w_1 = 1.$$

A gradient check by central differences (Note 12, Section 4; Note 7, Section 6.5, for the error of the formula) with $h = 10^{-6}$ reproduces all seven numbers to eight digits. The step $\mathbf{w} \leftarrow \mathbf{w} - \eta \, \partial \ell / \partial \mathbf{w}$ with $\eta = 0.1$ moves the kernel to $(1.8, -0.4)$.

Algorithm 3 Convolutional layer, backward pass (stride 1, one example)

Require: stored padded input $\bar{x}$ and pre-activations z from Algorithm 1; weights W; gradient $G = \partial \ell / \partial h \in \mathbb{R}^{C_{\text{out}} \times H' \times W'}$

function ConvBackward(G; $\bar{x}, z, W$)
  $\Delta \leftarrow G \odot g'(z)$ ▷ through the activation
  $\partial \ell / \partial b_c \leftarrow \sum_{u,v} \Delta_{c,u,v}$ for each c
  for c, c', a, b **do**
    $\partial \ell / \partial W_{c,c',a,b} \leftarrow \sum_{u,v} \Delta_{c,u,v}\, \bar{x}_{c',\, u+a,\, v+b}$ ▷ input ⋆ gradient
  end for
  $\Gamma \leftarrow$ zero array of the shape of $\bar{x}$
  for c, u, v **do** ▷ scatter each output gradient over its window
    $\Gamma_{c',\, u+a,\, v+b}$ += $\Delta_{c,u,v}\, W_{c,c',a,b}$ for all c', a, b
  end for
  return $\partial \ell / \partial W$, $\partial \ell / \partial \mathbf{b}$, and $\partial \ell / \partial x = \Gamma$ without its padding border
end function

Running time — convolutional layer, backward

Time: the weight gradient and the input gradient each need $C_{\text{out}} H' W' C_{\text{in}} k^2$ multiply–adds, the same count as the forward pass, so one training step costs about three forward passes; the input gradient of the first layer is not needed and can be skipped. *Memory*: the stored input and pre-activations, $O(C_{\text{in}} HW + C_{\text{out}} H' W')$ per example, times the mini-batch size n'; gradients of the weights need as much memory as the weights. *Training* a network with F multiply–adds per forward pass on n examples for E epochs therefore costs about $3FnE$ multiply–adds. With a stride $s > 1$ both the weight-gradient sum and the scatter step use the positions $su + a$, $sv + b$ instead of $u + a$, $v + b$.

One-by-one convolutions. A convolution with $k = 1$ looks at a single position: it is a fully connected layer from C_{in} to C_{out} channels, applied at every pixel with shared weights. It mixes channels without mixing positions, and is used to change the number of channels cheaply. Lin, Chen and Yan's *network in network* replaced each linear filter by a small MLP applied at every position, which is the same as stacking 1×1 convolutions [23]; GoogLeNet [35] and the residual bottlenecks of Section 7 use them to reduce channels before expensive 3×3 convolutions.

6 Data augmentation: learning invariances from transformed copies

Convolution and pooling build in equivariance and a little invariance to *small translations*. Other transformations that leave the label unchanged — rotations, changes of scale, reflections, illumination, the elastic wobble of handwriting — are not built in. A simple and very effective remedy is to *generate* them: apply random label-preserving transformations to the training images and train on the transformed copies, usually drawing a fresh transformation every time an image is visited. This is *data augmentation*. The network then learns the invariance from examples; at the same time augmentation acts as a regulariser (Note 4, Section 7.3), because the effective training set is larger and more varied.

Typical transformations for natural images are random crops (with the crop rescaled to the input size), horizontal flips, small rotations and scalings, and perturbations of brightness, contrast and colour. Which transformations are allowed depends on the task: a horizontal flip preserves "cat" but turns a "b" into a "d", and a 180° rotation turns a "6" into a "9". Augmentation encodes knowledge about the problem, and wrong knowledge hurts.

Elastic distortions. For handwritten characters, Simard, Steinkraus and Platt proposed *elastic distortions* [33]. Draw an independent random displacement $(\Delta u, \Delta v) \in [-1,1]^2$ for every pixel; this field is pure noise. Smooth it by convolving each component with a Gaussian of width σ pixels (yet another convolution), and multiply it by an intensity α. The new image takes at each pixel (u, v) the value of the old image at $(u + \Delta u, v + \Delta v)$, read by bilinear interpolation between the four nearest pixels: for a point $(u_0 + a, v_0 + b)$ with integer u_0, v_0 and $0 \le a, b < 1$ it returns $(1-a)(1-b)\, x_{u_0,v_0} + a(1-b)\, x_{u_0+1,v_0} + (1-a)b\, x_{u_0,v_0+1} + ab\, x_{u_0+1,v_0+1}$. The width σ sets how coherent the displacements are, α how large: small σ gives a noisy, torn image, a moderate σ a plausible variant of the same character, and a large σ an almost uniform shift or shear (Figure 3).

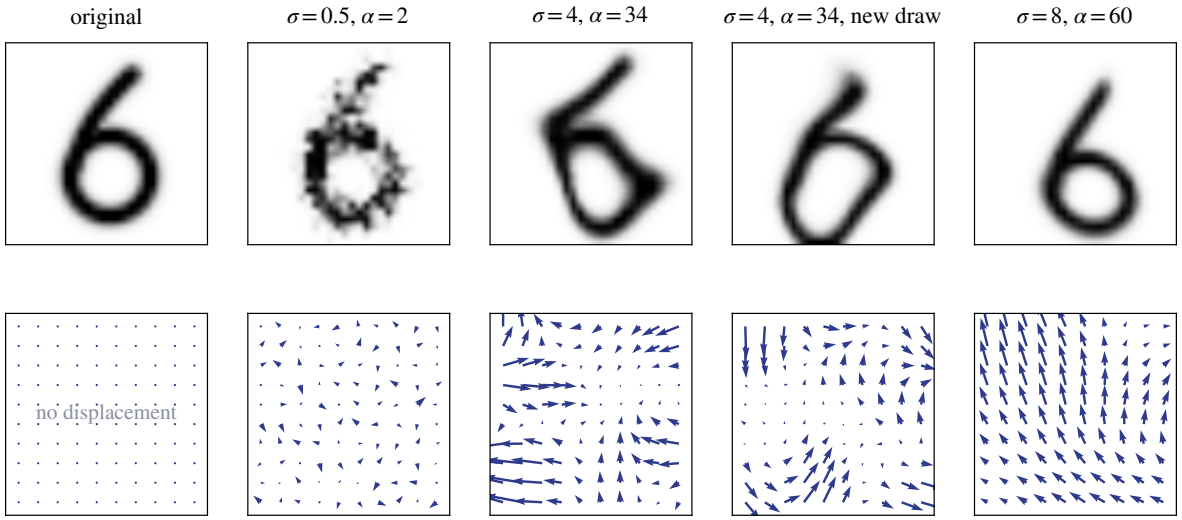


Figure 3. Elastic distortions of a synthetic 28×28 digit (top) and the displacement fields that produced them (bottom, every third pixel, arrows drawn to scale). With $\sigma = 0.5$ the field is nearly independent from pixel to pixel and tears the stroke; with $\sigma = 4$ and $\alpha = 34$ it bends the digit coherently (root-mean-square displacement 2.6 pixels in the first draw); two draws give two different, plausible sixes. With $\sigma = 8$ the field is nearly uniform over the image.

Algorithm 4 Elastic distortion of an image

Require: image $x \in \mathbb{R}^{H \times W}$ (one channel; apply the same field to every channel); width σ; intensity α

function ElasticDistort($x; \sigma, \alpha$)
  draw $\Delta u_{u,v}, \Delta v_{u,v} \sim$ Uniform$[-1, 1]$ independently for every pixel
  $\Delta u \leftarrow \alpha\,(G_\sigma * \Delta u)$; $\Delta v \leftarrow \alpha\,(G_\sigma * \Delta v)$ ▷ Gaussian smoothing, separable, $O(HW\sigma)$
  for every pixel (u, v) **do**
    $x'_{u,v} \leftarrow$ Bilinear($x;\ u + \Delta u_{u,v},\ v + \Delta v_{u,v}$) ▷ weighted mean of four neighbours
  end for
  return x' ▷ called anew each time an image enters a mini-batch
end function

Running time — elastic distortion and on-the-fly augmentation

Time: $O(HW\sigma)$ per image for the separable smoothing (two one-dimensional Gaussian filters of length $\approx 6\sigma$ per field component) and $O(HW)$ for the interpolation — small compared with a forward pass through the network, and it can run on the CPU while the GPU trains. *Memory*: $O(HW)$ for the field; augmenting on the fly stores no extra examples, whereas precomputing A transformed copies of n images would need A times the storage. *Training time* per epoch is unchanged, though more epochs are usually needed. *Prediction*: no cost, unless predictions are averaged over several augmented copies of the test image (test-time augmentation), which multiplies the prediction cost by the number of copies.

Modern connection — augmentation in current practice

The standard ImageNet recipe since AlexNet uses random crops and horizontal flips; modern pipelines add random resizing, colour jitter and policies chosen automatically. *Mixup* [40] goes beyond label-preserving transformations: it trains on convex combinations $\lambda\mathbf{x}_i+(1-\lambda)\mathbf{x}_j$ of two inputs with the same combination of their one-hot labels. In self-supervised learning augmentation even defines the task: SimCLR [3] trains a network to map two random augmentations of the same image to nearby representations and those of different images far apart, with no labels at all. The choice of augmentations then determines which invariances the representation acquires.

In physics — jet images: augment or standardise

At the Large Hadron Collider the energy deposits of a jet in the calorimeter can be drawn as an image on the (pseudorapidity, azimuth) plane, with pixel intensities equal to the transverse energy. Cogan et al. introduced such jet images [4], and convnets trained on them separate jets from boosted W bosons from ordinary quark and gluon jets [7]. Some symmetries are exact here — a shift in azimuth, a rotation about the jet axis to good approximation — and there are two ways to use them. One can augment, showing the network randomly rotated and translated copies; or one can remove the nuisance variation by preprocessing, centring every jet on its leading subjet and rotating it into a standard orientation, as these studies do. Preprocessing is cheaper and exact when the symmetry is; augmentation is the only option when no canonical orientation exists; equivariant architectures (Section 3.3) are a third option.

7 From AlexNet to residual networks

AlexNet. The ImageNet Large Scale Visual Recognition Challenge (ILSVRC) asks for one of 1000 classes for each image and provides about 1.2 million labelled training images [6]. In 2012 Krizhevsky, Sutskever and Hinton entered a convnet with five convolutional and three fully connected layers, 60 million parameters and 650 000 neurons [20] (the timeline of Note 11, Section 8, places it in context). Trained on two GPUs for about a week, it reached a top-5 test error of 15.3% (the true class is not among the five highest-scoring guesses), against 26.2% for the best entry built on hand-designed features. Its ingredients were few and are all standard now: ReLU activations, which trained several times faster than saturating units (and variants followed, such as maxout units, the maximum of several linear functions [16]); dropout in the fully connected layers (Note 12, Section 6.3); heavy data augmentation (Section 6); and a fast GPU implementation of convolutions. The result ended the era of hand-designed visual features; within a few years almost all entries were convnets.

What the filters learn. The first-layer filters of a trained convnet can be displayed directly as small images. They are oriented edge detectors at various scales and frequencies, and colour-contrast blobs — strikingly similar to the receptive fields Hubel and Wiesel found, and to the Gabor filters designed by hand before. Deeper units are not filters on pixels and must be visualised indirectly: by the training patches that excite them most, or by projecting their activation back to pixel space. Zeiler and Fergus did the latter with a "deconvolutional" network built from the transposed operations of Section 10 [39]. They found a hierarchy: corners and conjunctions of edges in the second layer, textures and repeated patterns in the third, object parts such as dog faces and bird legs in the fourth, and whole objects in varied poses in the fifth. The visualisations also diagnosed problems — aliasing caused by the large first-layer stride — and a network with smaller first-layer filters and

stride performed better.

Deeper, with small filters. VGG networks [34] used only 3×3 convolutions, stacked 16 or 19 weight layers deep, and showed that depth itself improves accuracy. GoogLeNet [35] combined 1×1, 3×3 and 5×5 convolutions in parallel "inception" modules.

> **Worked example — 8. Why small filters, and why bottlenecks**
>
> *Stacking.* Two 3×3 layers in a row see a 5×5 patch, by (7): $1 + 2 + 2 = 5$. With C channels in and out (biases ignored) they have $2 \times 9C^2 = 18C^2$ weights, against $25C^2$ for one 5×5 layer; three 3×3 layers see 7×7 with $27C^2$ weights against $49C^2$. For $C = 256$: 1 179 648 against 1 638 400, and 1 769 472 against 3 211 264. The stack also inserts a non-linearity between the layers, so it is cheaper *and* more expressive.
> *Bottleneck.* A residual block of the deeper ResNets (below) acting on 256 channels uses 1×1 convolutions to go down to 64 channels, a 3×3 convolution at 64 channels, and a 1×1 convolution back up to 256:
>
> $$256 \cdot 64 + 9 \cdot 64 \cdot 64 + 64 \cdot 256 = 16\,384 + 36\,864 + 16\,384 = 69\,632$$
>
> weights, against $2 \times 9 \times 256^2 = 1\,179\,648$ for two 3×3 convolutions at 256 channels — seventeen times fewer, at the same input and output width.

Residual networks. Making plain convnets deeper eventually made them *worse*, on the training set as well as on the test set, which is an optimisation problem, not overfitting. In the residual networks of He, Zhang, Ren and Sun [13] each block adds a learned correction to its input,

$$\mathbf{h}_{\text{out}} = \mathrm{ReLU}\big(\mathbf{h} + F(\mathbf{h})\big), \tag{10}$$

where F is two or three convolutions with batch normalisation (Note 12, Section 10.2) and ReLUs in between; why this keeps very deep stacks trainable is derived in Note 12, Section 10.3. Two details are specific to convnets. When F changes the number of channels or uses stride 2, the shortcut becomes a 1×1 convolution with the same stride, so that the shapes match. And the original block applies the ReLU *after* the addition, unlike the pre-normalised form of Note 12, Section 10.3. With this design networks of 152 layers trained well, and an ensemble of them won ILSVRC 2015 with a top-5 error of 3.57%; an often-quoted estimate of trained human top-5 error on the same task is about 5% [32].

Algorithm 5 Residual block of a convnet (basic form)

Require: feature maps $h \in \mathbb{R}^{C \times H \times W}$; two 3×3 convolutions with C' output channels, the first with stride s; normalisation layers N_1, N_2

function ResidualBlock(h)
  $u \leftarrow \mathrm{ReLU}\big(N_1(\mathrm{Conv}_{3\times3,\,s}(h))\big)$ ▷ Algorithm 1, padding 1
  $u \leftarrow N_2(\mathrm{Conv}_{3\times3,\,1}(u))$
  if $C' \neq C$ or $s \neq 1$ **then**
    $h \leftarrow \mathrm{Conv}_{1\times1,\,s}(h)$ ▷ projection shortcut, matches the shape of u
  end if
  return $\mathrm{ReLU}(h + u)$ ▷ add, then activate
end function

Running time — residual block

Time: two 3×3 convolutions, $9C'(C+C')H'W'$ multiply–adds per example, plus $CC'H'W'$ for a projection shortcut and $O(C'H'W')$ for the addition, normalisations and activations. The identity shortcut is free. *Memory*: $9C'(C+C')$ weights (plus CC' for a projection); activations of both convolutions are stored for the backward pass. *Backward*: about twice the forward cost; the shortcut adds no cost (Note 12, Section 10.3).

Attention to channels. Squeeze-and-excitation blocks [17] add a cheap form of global context: average each of the C channels over all positions to a vector $\mathbf{s} \in \mathbb{R}^C$, pass it through a small two-layer network ending in a sigmoid to obtain channel weights $\mathbf{g} \in (0,1)^C$, and multiply channel c by g_c. With a reduction from C to C/r hidden units this costs $2C^2/r$ weights: 8192 for $C = 256$ and $r = 16$. An ensemble of SE networks won the final ILSVRC, in 2017, with a top-5 error of 2.25% [17]. Algorithm 6 also gives the *depthwise separable* convolution of the box below, which factorises a $k \times k$ layer into a per-channel spatial filter and a 1×1 channel mixer.

Algorithm 6 Squeeze-and-excitation block and depthwise separable convolution

Require: feature maps $h \in \mathbb{R}^{C\times H\times W}$; $W_1 \in \mathbb{R}^{(C/r)\times C}$, $W_2 \in \mathbb{R}^{C\times(C/r)}$, biases $\mathbf{b}_1, \mathbf{b}_2$

function SqueezeExcite(h)
  $s_c \leftarrow \frac{1}{HW}\sum_{u,v} h_{c,u,v}$ for each c ▷ squeeze: global average pooling
  $\mathbf{g} \leftarrow \text{sigmoid}\left(W_2\ \text{ReLU}(W_1\mathbf{s} + \mathbf{b}_1) + \mathbf{b}_2\right)$ ▷ excite: $C \to C/r \to C$
  return $h'_{c,u,v} = g_c\, h_{c,u,v}$ ▷ rescale each channel
end function
function DepthwiseSeparable(x; $k \times k$ kernels $D_{c'}$, 1×1 weights $V \in \mathbb{R}^{C_{\text{out}}\times C_{\text{in}}}$)
  $q_{c'} \leftarrow x_{c'} \star D_{c'}$ for each input channel c' ▷ depthwise: $C_{\text{in}}k^2$ weights
  return $z_{c,u,v} = \sum_{c'} V_{c,c'}\, q_{c',u,v}$ ▷ pointwise: $C_{\text{in}}C_{\text{out}}$ weights
end function

Running time — squeeze-and-excitation and depthwise separable convolution

SE time: $O(CHW)$ for the squeeze and for the rescaling (each entry is read once), $O(2C^2/r)$ for the two small layers — negligible next to the convolutions around it. *SE memory*: $2C^2/r$ weights plus $C/r + C$ biases. *Depthwise separable time*: $O(C_{\text{in}}H'W'k^2)$ for the per-channel filters plus $O(C_{\text{in}}C_{\text{out}}H'W')$ for the channel mixing, instead of $O(C_{\text{in}}C_{\text{out}}H'W'k^2)$: a saving of almost k^2 when $C_{\text{out}} \gg k^2$. *Memory*: $C_{\text{in}}(k^2+1) + C_{\text{out}}(C_{\text{in}}+1)$ parameters with biases. *Prediction*: the same as the forward pass.

Modern connection — patches, transformers and convnets today

The vision transformer (ViT) [8] cuts a 224×224 image into 16×16 patches and maps each to a vector of dimension D by one shared linear map. That patch embedding *is* a convolution with kernel size 16 and stride 16: by Worked example 10 it produces a $14 \times 14 = 196$-position map with D channels, and for $D = 768$ it has $16 \cdot 16 \cdot 3 \cdot 768 + 768 = 590\,592$ parameters. Everything after it is a transformer: attention (Note 5, Section 3.3) lets every patch interact with every other from the first layer on, and locality survives only in the patch embedding. Without this prior ViT needs more data: trained on ImageNet alone it falls behind comparable ResNets, while pre-trained on far larger data sets it matches or surpasses them. ConvNeXt [24] then modernised the ResNet with large 7×7 depthwise convolutions (the separable design of MobileNets [15], Algorithm 6), layer normalisation (Note 12, Section 10.2) and GELU (Note 11, Section 2), and competes with vision transformers. Locality and sharing remain good priors; how much of them to hard-wire depends on how much data is available.

8 Convolution arithmetic

This section collects the bookkeeping that every practical convnet needs: the matrix form of a convolution, the size of its output, padding, stride and dilation, and inputs of variable size. We follow the conventions of Dumoulin and Visin [9]: along one spatial axis, i is the input size, k the kernel size, s the stride, the step by which the window advances, p the number of zeros appended on *both* sides of the axis, and o the output size. Axes are treated independently, so non-square images and kernels need nothing new.

8.1 Convolution is a sparse matrix product

Unroll the input into a vector (rows one after the other) and the output likewise. A convolution is linear in its input, so $\mathbf{y} = C\mathbf{x}$ for some matrix C. Row j of C holds the kernel at the positions of window j and zeros elsewhere.

Worked example — 9. A convolution written as a matrix

One dimension. The convolution of Worked example 2, $i = 5$, $k = 3$, $s = 1$ and $w = (1, 0, -1)$, reads

$$\begin{pmatrix} y_0 \\ y_1 \\ y_2 \end{pmatrix} = \begin{pmatrix} 1 & 0 & -1 & 0 & 0 \\ 0 & 1 & 0 & -1 & 0 \\ 0 & 0 & 1 & 0 & -1 \end{pmatrix} \begin{pmatrix} x_0 \\ x_1 \\ x_2 \\ x_3 \\ x_4 \end{pmatrix},$$

and with $x = (2, 3, 5, 4, 1)$ it returns $(-3, -1, 4)$, as before. Each row is the previous one shifted right by s (a *Toeplitz* structure, constant along diagonals). With stride 2 only every second row would be kept.

Two dimensions. For a 5×5 input and a 3×3 kernel, C is 9×25: each of the nine rows contains the nine kernel entries, arranged in three groups of three separated by two zeros (the jump to the next image row), and zeros elsewhere — 81 non-zeros out of 225 entries, or 36%. Here C is fairly dense only because the image is tiny. For a 224×224 image, a 3×3 kernel and an output of the same size, C is $50\,176 \times 50\,176$ with at most 9 non-zeros per row, a fraction $9/50\,176 \approx 1.8 \times 10^{-4}$ of its entries, and only 9 distinct values.

The matrix view explains two facts at once. First, a convolutional layer is a fully connected layer whose weight matrix is constrained to be sparse (local connectivity) and to repeat the same values (sharing); everything known about MLPs applies. Second, by the chain rule $\partial \ell / \partial \mathbf{x} = C^{\mathsf{T}}\, \partial \ell / \partial \mathbf{y}$: the backward pass multiplies by the *transpose*. Equation (9) is exactly multiplication by C^{T}; Section 10 uses C^{T} as a layer in its own right.

The im2col form. Libraries rarely loop as in Algorithm 1. They copy every receptive field, a $C_{\text{in}}k^2$ vector, into one row of a *patch matrix* P with $H'W'$ rows ("image to columns"), and compute all outputs with a single dense matrix product $P\,\tilde{W}^{\mathsf{T}}$, where $\tilde{W} \in \mathbb{R}^{C_{\text{out}} \times C_{\text{in}}k^2}$ holds one flattened filter per row. Dense matrix products run close to the peak speed of GPUs, which more than pays for the copying.

Algorithm 7 Convolution as one matrix product (im2col)

Require: padded input $\bar{x} \in \mathbb{R}^{C_{\text{in}} \times (H+2p) \times (W+2p)}$; weights W; biases $\mathbf{b}$; stride s

function Im2colConv($\bar{x}; W, \mathbf{b}, s$)
  $P \leftarrow$ empty matrix with $H'W'$ rows and $C_{\text{in}}k^2$ columns
  for $u = 0, \dots, H'-1$; $v = 0, \dots, W'-1$ **do**
    row $(uW'+v)$ of $P \leftarrow \bar{x}_{:,\, su:su+k,\, sv:sv+k}$ flattened ▷ copy, no arithmetic
  end for
  $\tilde{W} \leftarrow W$ reshaped to $C_{\text{out}} \times C_{\text{in}}k^2$
  $Z \leftarrow P\tilde{W}^{\mathsf{T}} + \mathbf{1}\mathbf{b}^{\mathsf{T}}$ ▷ one GEMM, $H'W' \times C_{\text{out}}$
  return Z reshaped to $C_{\text{out}} \times H' \times W'$
end function

Running time — im2col convolution

Time: $O(H'W'C_{\text{in}}k^2)$ to build P and $O(H'W'C_{\text{in}}k^2C_{\text{out}})$ for the product, the same multiply–add count as the direct form but executed as one highly optimised matrix multiplication. *Memory*: P duplicates every input value up to k^2 times: for 64 channels on a 56×56 map with $k = 3$ it holds 1.8×10^6 numbers (7.2 MB in 32-bit floats) against 0.2×10^6 for the map itself. Alternatives trade differently: FFT-based convolution costs $O(HW \log(HW))$ per pair of channels and pays off for large kernels, and specialised small-kernel algorithms reduce the multiplication count further for 3×3 filters.

8.2 Output size, padding and stride

Derivation of the output size. After padding, the axis has $i + 2p$ entries, numbered $0, \dots, i + 2p - 1$. The kernel starts at positions $0, s, 2s, \dots$; a start js is allowed if the window ends inside the axis, $js + k - 1 \le i + 2p - 1$, that is $j \le (i + 2p - k)/s$. Counting $j = 0, 1, \dots$ up to the largest such integer gives

$$o = \left\lfloor \frac{i + 2p - k}{s} \right\rfloor + 1. \tag{11}$$

When s does not divide $i + 2p - k$, the last $(i + 2p - k) \bmod s$ positions of the padded axis lie in no window and are ignored. For example $i = 7$ and $i = 8$ both give $o = 4$ with $k = 3$, $s = 2$, $p = 1$; and with $p = 0$ the entry x_7 of a length-8 signal is never used ($o = 3$).

Three kinds of padding. Without padding the output shrinks by $k - 1$ in each layer, and border pixels take part in fewer windows than central ones. Three choices are common for $s = 1$ (Figure 4):

- *valid* ($p = 0$): only complete windows, $o = i - k + 1$;
- *same* or *half* ($p = (k-1)/2$ for odd k): $o = i$, so that layers can be stacked without shrinking — the usual choice, as in the 3×3, padding-1 layers of VGG and ResNet;
- *full* ($p = k - 1$): every window that overlaps the input in at least one entry, $o = i + k - 1$. This is the length of the full convolution of (9), and of the product of two polynomials of degrees $i - 1$ and $k - 1$.

Zero padding is the default. *Reflect* padding (mirror the border) and *circular* padding (wrap around) are alternatives; the latter is the natural choice for periodic data.

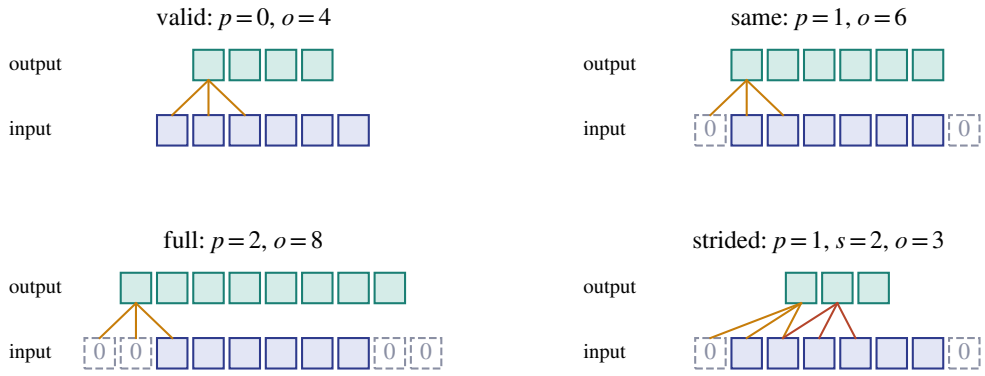


Figure 4. Padding and stride along one axis for $i = 6$ and $k = 3$; dashed cells are the padding zeros. The lines show the window of the first output (and, for stride 2, of the second). Valid, same and full padding give $o = 4, 6, 8$; with $p = 1$ and $s = 2$, $o = \lfloor 5/2 \rfloor + 1 = 3$.

Worked example — 10. Output sizes

By (11), with the arguments (i, k, s, p) (and k replaced by $\hat{k}$ of (12) for a dilation $d > 1$):

(i, k, s, p)	computation	o
$(6, 3, 1, 0)$	$\lfloor 3/1 \rfloor + 1$	4
$(6, 3, 1, 1)$, same	$\lfloor 5/1 \rfloor + 1$	6
$(6, 4, 1, 2)$, even kernel	$\lfloor 6/1 \rfloor + 1$	7
$(7, 3, 2, 1)$	$\lfloor 6/2 \rfloor + 1$	4
$(7, 3, 2, 0)$	$\lfloor 4/2 \rfloor + 1$	3
$(10, 3, 1, 0)$, $d = 3$, $\hat{k} = 7$	$\lfloor 3/1 \rfloor + 1$	4
$(32, 3, 1, 2)$, $d = 2$, $\hat{k} = 5$, same	$\lfloor 31/1 \rfloor + 1$	32
$(224, 7, 2, 3)$, first layer of a ResNet	$\lfloor 223/2 \rfloor + 1$	112
$(227, 11, 4, 0)$, AlexNet-like first layer	$\lfloor 216/4 \rfloor + 1$	55
$(224, 16, 16, 0)$, ViT patches	$\lfloor 208/16 \rfloor + 1$	14

An even kernel cannot be centred, so "same" padding with $k = 4$ needs unequal padding on the two sides (1 and 2); $p = 2$ on both sides gives one extra output. Pooling obeys the same formula: 2×2 pooling with stride 2 on a 28-pixel axis gives $\lfloor 26/2 \rfloor + 1 = 14$.

8.3 Dilation

A *dilated* (or atrous) convolution spreads the k kernel entries out, with $d-1$ skipped positions between consecutive entries:

$$y_j = \sum_{m=0}^{k-1} w_m \, x_{j+d\,m}.$$

The dilation $d = 1$ is an ordinary convolution. The kernel still has k weights but spans

$$\hat{k} = k + (k - 1)(d - 1) \tag{12}$$

input positions, and the output size is (11) with k replaced by $\hat{k}$ (two rows of the table of Worked example 10). Dilation enlarges the receptive field without extra weights, without extra computation and without the loss of resolution that pooling or strides cause; it is used for dense prediction tasks such as segmentation [38] and, with exponentially growing d, in the audio models of Section 9.

8.4 Inputs of variable size

Convolutions and pooling accept any input size; only their output size changes. The fully connected layers at the end of LeNet or AlexNet do not: they need a fixed number of inputs. Two remedies are common. Pooling regions can be made proportional to the input, so that the pooled map always has the same size. More simply, *global average pooling* averages each of the last C feature maps over all its positions and passes the resulting C-vector to the classifier, whatever the image size [23]; it also removes the parameter-heavy fully connected layers (compare the table of Worked example 6). A network made only of convolutions, pooling and pointwise operations is called *fully convolutional*; it maps an image of any size to a map of outputs, which Section 10 exploits.

> **In physics — periodic lattices and circular padding**
>
> Monte Carlo configurations of a lattice model, such as the two-dimensional Ising model on an $L \times L$ lattice with periodic boundary conditions, are natural inputs for a convnet: the Hamiltonian is local and translation invariant, which are exactly the assumptions of Section 2. Carrasquilla and Melko showed that neural networks trained on such configurations can classify the ordered and disordered phases and locate the transition [2]. With periodic boundaries there is no edge, so zero padding would invent one; with stride-1 layers and no intermediate pooling, circular padding makes every layer exactly equivariant under lattice translations, and a final global average pool makes the prediction exactly invariant (with strides or pooling, only under shifts by multiples of the total stride). A network built this way can be trained on small lattices and applied, unchanged, to larger ones — a useful check of finite-size scaling.

9 Causal dilated stacks: WaveNet and ByteNet

Raw audio has structure on many time scales at once: individual oscillations last a fraction of a millisecond, phonemes tens of milliseconds, words and notes much longer. WaveNet [37] generates audio one sample at a time, at 16 000 samples per second, predicting each sample from the samples before it. Two kinds of one-dimensional convolution make this possible.

Causal convolutions. For generation the output at time t may depend only on inputs at times $\le t$. A *causal* convolution achieves this by padding only on the left, with $\hat{k}-1$ zeros: the output keeps the length of the input and y_t depends on $x_{t-\hat{k}+1}, \dots, x_t$.

Exponentially dilated stacks. With kernel size $k = 2$ and dilations $1, 2, 4, \dots, 2^{L-1}$ in successive layers, each layer adds $d_l(k-1)$ to the receptive field, so by (7) (strides 1, kernels of span $\hat{k}_l = d_l + 1$)

$$r = 1 + \sum_{l=1}^{L} d_l = 1 + (1 + 2 + \cdots + 2^{L-1}) = 2^L.$$

The receptive field doubles with every layer, while the number of weights and the cost grow only linearly in L (Figure 5).

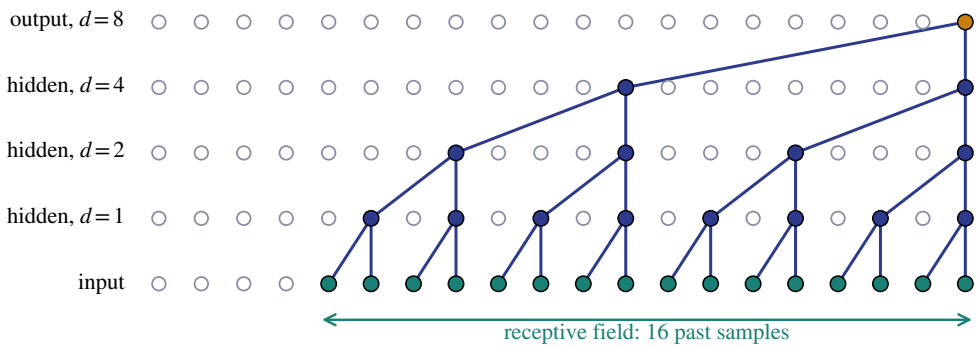


Figure 5. A stack of causal convolutions with kernel size 2 and dilations $1, 2, 4, 8$. The highlighted connections feed the last output; it depends on exactly the $2^4 = 16$ most recent inputs, with 4 layers of 2 weights each.

Worked example — 11. Receptive fields of WaveNet

WaveNet stacks blocks with dilations $1, 2, 4, \dots, 512$ ($L = 10$ layers, $k = 2$). One block has receptive field $2^{10} = 1024$ samples, 64 ms at 16 kHz. Repeating the block three times adds 1023 samples per block: $1 + 3 \times 1023 = 3070$ samples, about 0.19 s. Reaching 1024 samples with undilated $k = 2$ convolutions would take 1023 layers; with $k = 3$ undilated, 512 layers.

The audio model also uses gated activations $\tanh(\cdot) \odot \sigma(\cdot)$, residual and skip connections, and a softmax over 256 quantised amplitude levels for each sample — classification, sample by sample. The same causal dilated stacks were used for machine translation (ByteNet [19]), before transformers took over that role.

Algorithm 8 Stack of dilated causal convolutions (one channel per layer for clarity)

Require: sequence $x_0, \dots, x_{T-1}$; layers $l = 1, \dots, L$ with weights $w^{(l)} \in \mathbb{R}^k$, bias $b^{(l)}$, dilation d_l; activation g

function DilatedCausalStack(x)
  $h^{(0)} \leftarrow x$
  for $l = 1, \dots, L$ **do**
    for $t = 0, \dots, T-1$ **do**
      $h_t^{(l)} \leftarrow g\big(b^{(l)} + \sum_{m=0}^{k-1} w_m^{(l)}\, h_{t-d_l(k-1-m)}^{(l-1)}\big)$ ▷ $h_{t'} = 0$ for $t' < 0$: left padding
    end for
  end for
  return $h^{(L)}$ ▷ depends on $x_{t-r+1}, \dots, x_t$ only, $r = 1 + (k-1)\sum_l d_l$
end function

Running time — dilated causal stack

Time: $O(TLkC^2)$ for T time steps, L layers and C channels per layer, the same as undilated convolutions; the receptive field $1 + (k-1)\sum_l d_l$ grows exponentially in L for $d_l = 2^{l-1}$. *Training* is parallel over all T positions because the whole target sequence is known. *Generation* is sequential: each new sample needs a pass through all L layers, $O(LkC^2)$ per sample if the past activations of every layer are cached (otherwise $O(rLkC^2)$), and 16 000 such passes per second of audio. *Memory*: LkC^2 weights; the cache needs $(k-1)d_l$ past values per layer and channel.

In physics — one-dimensional convnets on detector time series

Many physics signals are long one-dimensional series in which a short, characteristic pattern must be found anywhere: a gravitational-wave chirp in interferometer strain data, a pulse in a photomultiplier trace, a transit in a stellar light curve. Translation equivariance in time is the right prior, and the receptive field must cover the longest pattern. George and Huerta trained one-dimensional convnets directly on simulated detector strain to detect binary black-hole mergers and estimate their parameters [11], an alternative to matched filtering against a bank of templates that is far cheaper at prediction time.

10 Structured outputs: transposed convolutions and upsampling

In *semantic segmentation* every pixel receives a class label (road, car, pedestrian; tumour or healthy tissue); in image generation, super-resolution or the prediction of a physical field, the output is itself an image. A fully convolutional network [25] can produce such outputs: an *encoder* of convolutions and pooling reduces the image to a small, deep feature map, and a *decoder* enlarges it back to the input resolution, ending in a per-pixel softmax or regression. The decoder needs a layer that *increases* spatial size.

A parameter-free option is *max-unpooling* [39]: the decoder puts each value at the position recorded by the matching encoder's max-pooling layer (the indices arg of Algorithm 2) and fills the rest of the block with zeros, exactly what MaxPoolBackward does with a gradient. It restores *where* features were, not what surrounded them, and is followed by convolutions. Two learnable layers are in use as well.

10.1 Transposed convolution

A convolution maps $\mathbf{x} \in \mathbb{R}^i$ to $\mathbf{y} = C\mathbf{x} \in \mathbb{R}^o$, with $o \le i$ for the usual choices $p \le (k-1)/2$. The *transposed convolution* uses the same sparse structure in the other direction, $\mathbf{x}' = C^{\mathsf{T}}\mathbf{y}'$: from o values to i. Read column by column, $C^{\mathsf{T}}\mathbf{y}' = \sum_j y'_j$ (row j of $C)^{\mathsf{T}}$: each input value y'_j *scatters* a scaled copy of the kernel onto the output, at position js, and overlapping copies add. As Section 8.1 showed, this is the backward pass of a convolution, now used in the forward direction with learned weights. It is not an inverse: $C^{\mathsf{T}}C \neq I$ in general. The name "deconvolution" is common but misleading; "fractionally strided convolution" describes an equivalent implementation: insert $s-1$ zeros between consecutive input values, pad with $k-1-p$ zeros on each side, and cross-correlate with the flipped kernel at stride 1 — equivalent, but wasteful, since most multiplications are by zero.

The output size inverts (11). If s divides $i+2p-k$, (11) reads $o-1 = (i+2p-k)/s$; solving for i, a transposed convolution with parameters (k, s, p) maps o entries to

$$i' = s(o-1) + k - 2p, \tag{13}$$

the input size of the corresponding convolution when s divides $i+2p-k$. Otherwise several input sizes share the same o (Section 8), and libraries take an extra "output padding" argument to choose among them.

Worked example — 12. A transposed convolution by hand, and its checkerboard

Scatter. Input $\mathbf{y}' = (1, 2, -1)$, kernel $w = (1, 2, 1)$, stride 2, no padding. By (13) the output has $2 \cdot 2 + 3 = 7$ entries. Each input value places a scaled kernel, two positions further along each

time:

$$(1,2,1,0,0,0,0)+(0,0,2,4,2,0,0)+(0,0,0,0,-1,-2,-1)=(1,2,3,4,1,-2,-1).$$

Check with the matrix. The convolution with $i=7$, $k=3$, $s=2$ has $o=3$ and

$$C=\begin{pmatrix}1&2&1&0&0&0&0\\0&0&1&2&1&0&0\\0&0&0&0&1&2&1\end{pmatrix},\qquad C^{\mathsf{T}}(1,2,-1)^{\mathsf{T}}=(1,2,3,4,1,-2,-1)^{\mathsf{T}}.$$

Check with zero insertion. $(1,2,-1)$ becomes $(1,0,2,0,-1)$, padded with $k-1-p=2$ zeros on each side; cross-correlating with the flipped kernel, here again $(1,2,1)$, gives the same seven numbers, with only 9 of the 21 products non-zero.
Uneven overlap. Feed a constant input $(1,1,1,1)$ and the kernel $(1,1,1)$ with stride 2. The output is $(1,1,2,1,2,1,2,1,1)$: positions reached by two copies of the kernel get 2, the others 1, although the input has no structure at all. With $k=4$ and $s=2$ the copies overlap evenly, and the interior of the output is constant, $(\dots,2,2,2,\dots)$. In two dimensions the pattern of a $k=3$, $s=2$ layer is the product of two such alternations, with interior values 1, 2 and 4: a checkerboard (Figure 6).

Algorithm 9 Transposed convolution (one input and one output channel)

Require: input $y'\in\mathbb{R}^{o_1\times o_2}$; kernel $w\in\mathbb{R}^{k\times k}$; stride s; padding p

function TransposedConv(y'; w, s, p)
  $\bar{x}\leftarrow$ zero array of size $(s(o_1-1)+k)\times(s(o_2-1)+k)$
  for $u=0,\dots,o_1-1$; $v=0,\dots,o_2-1$ **do**
    $\bar{x}_{su:su+k,\ sv:sv+k}$ += $y'_{u,v}\,w$ ▷ scatter a scaled kernel; overlaps add
  end for
  return $\bar{x}$ without p rows and columns on each side ▷ size $s(o-1)+k-2p$ per axis
end function

Running time — transposed convolution

Time: $O(o_1o_2k^2)$ per channel pair, so $O(C_{\text{in}}C_{\text{out}}o_1o_2k^2)$ for a layer — the cost of the convolution it transposes; the zero-insertion implementation wastes a factor of about s^2. *Memory*: $C_{\text{out}}(C_{\text{in}}k^2+1)$ parameters, as for a convolution; the output is about s^2 times larger than the input, which dominates memory in the high-resolution layers of a decoder. *Backward*: the gradient of a transposed convolution is an ordinary convolution, with the same cost.

10.2 Checkerboard artefacts and resize-convolution

When the kernel size is not divisible by the stride, the copies of the kernel overlap unevenly, and unless the weights happen to compensate — as the linear-interpolation kernel $(\frac{1}{2},1,\frac{1}{2})$ with $s=2$ does — outputs carry a periodic pattern even for perfectly smooth inputs (Worked example 12). A network can learn weights that cancel the pattern, but in practice it often does not quite, and images generated by such decoders show faint checkerboards. Odena, Dumoulin and Olah [28] recommended two remedies: choose k divisible by s (for instance $k=4$, $s=2$), or separate the two jobs — first *upsample* with a fixed interpolation (nearest neighbour or bilinear), then apply an ordinary stride-1 convolution. The second, *resize-convolution* (Algorithm 10), cannot produce an uneven overlap by construction and is now the usual choice in convolutional decoders that generate images (Figure 6).

Algorithm 10 Resize-convolution (upsample by s, then convolve)

Require: $y' \in \mathbb{R}^{C_{\text{in}} \times o_1 \times o_2}$; factor s; output $C_{\text{out}} \times so_1 \times so_2$; weights $W \in \mathbb{R}^{C_{\text{out}} \times C_{\text{in}} \times k \times k}$ (k odd), biases $\mathbf{b}$

function ResizeConv(y'; $s, W, \mathbf{b}$)
  $u_{c,\, su+a,\, sv+b} \leftarrow y'_{c,u,v}$ for all $0 \le a, b < s$ ▷ nearest neighbour: copy into $s \times s$ blocks
  return ConvForward(u; $W, \mathbf{b}$, stride 1, $p = (k-1)/2$) ▷ Algorithm 1
end function

Running time — resize-convolution

Time: $O(C_{\text{in}} s^2 o_1 o_2)$ for the upsampling (one copy per fine-grid entry) plus $O(C_{\text{out}} C_{\text{in}} s^2 o_1 o_2 k^2)$ for the convolution, which now runs on the fine grid — s^2 times the cost of a transposed convolution with the same k. *Memory*: $C_{\text{out}}(C_{\text{in}} k^2 + 1)$ parameters, as for any convolution, plus the upsampled intermediate $C_{\text{in}} s^2 o_1 o_2$, which is stored for the backward pass. *Prediction*: the same as the forward pass.

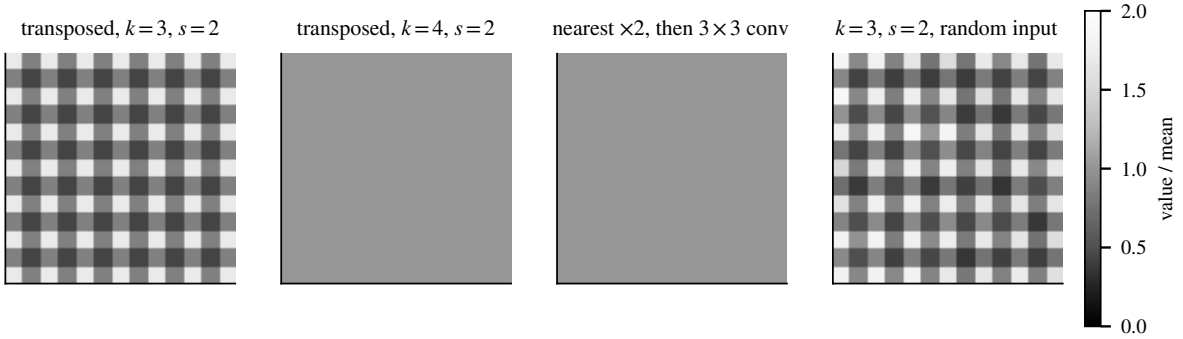


Figure 6. Upsampling an 8×8 constant input by a factor 2 (all kernel entries equal to one; borders cropped). A transposed convolution with $k = 3$ and $s = 2$ produces a checkerboard with values 1, 2 and 4; with $k = 4$ and $s = 2$ the output is uniform, as it is with nearest-neighbour upsampling followed by a 3×3 convolution. Right: the pattern of the $k = 3$ layer persists for a random input with values between 0.8 and 1.2.

Modern connection — U-Net, from microscopy to diffusion models

Pooling discards precise positions, and a decoder alone cannot recover them. The U-Net of Ronneberger, Fischer and Brox [31] adds *skip connections*: the feature maps of each encoder level are concatenated with the decoder maps of the same resolution, so that fine detail bypasses the bottleneck and coarse context comes up from below. Designed for segmenting cells in microscope images from very few annotated examples — with heavy elastic deformation as augmentation (Section 6) — it became the standard image-to-image architecture. Denoising diffusion models (Note 2, Section 5) [14] use a U-Net to predict the noise added to an image, and latent diffusion models [30] run such a U-Net on a compressed latent representation; many later systems replace it with a transformer, but the encoder–decoder-with-skips idea remains.

In physics — convolutional surrogates for field equations

Many simulations produce fields on grids: velocity and pressure around a body, a density field in cosmology. When the same kind of simulation must be run thousands of times, as in a design loop, an image-to-image network trained on a few thousand solved cases can act as a fast *surrogate*. Thuerey and co-workers trained U-Nets to predict the Reynolds-averaged velocity and pressure fields around airfoils from the geometry and flow conditions, far faster than the solver [36]. Local differential operators become local convolutions, and the encoder's coarse levels carry the long-range part of the solution, like the coarse grids of a multigrid solver.

11 Costs at a glance

Table 1 collects the costs of this note for one example, with C_{in} and C_{out} channels, $k \times k$ kernels, an input of $H \times W$ positions and an output of $H' \times W'$. Training costs about $3FnE$ for F forward multiply–adds (runtime box of Algorithm 3).

Table 1. Costs per example of the operations in this note (multiply–adds or comparisons).

operation	forward time	parameters	stored for backward
fully connected, $d \to m$	$O(dm)$	$m(d+1)$	$O(d+m)$
convolution (direct or im2col)	$O(C_{\text{out}}H'W'C_{\text{in}}k^2)$	$C_{\text{out}}(C_{\text{in}}k^2+1)$	$O(C_{\text{in}}HW + C_{\text{out}}H'W')$
1×1 convolution	$O(C_{\text{out}}C_{\text{in}}HW)$	$C_{\text{out}}(C_{\text{in}}+1)$	$O((C_{\text{in}}+C_{\text{out}})HW)$
depthwise separable, $k \times k$	$O(C_{\text{in}}HW(k^2+C_{\text{out}}))$	$C_{\text{in}}(k^2+1)+C_{\text{out}}(C_{\text{in}}+1)$	$O((C_{\text{in}}+C_{\text{out}})HW)$
squeeze-and-excitation	$O(CHW + C^2/r)$	$2C^2/r + C/r + C$	$O(CHW)$
max or average pooling	$O(CHW)$	0	$O(CHW/m^2)$ indices
dilated convolution	as convolution	as convolution	as convolution
max-unpooling	$O(CHW)$	0	—
transposed convolution	$O(C_{\text{out}}C_{\text{in}}o_1o_2k^2)$	$C_{\text{out}}(C_{\text{in}}k^2+1)$	$O(C_{\text{in}}o_1o_2)$
resize-convolution, $\times s$	$O(C_{\text{out}}C_{\text{in}}s^2o_1o_2k^2)$	$C_{\text{out}}(C_{\text{in}}k^2+1)$	$O(C_{\text{in}}s^2o_1o_2)$
residual block (basic)	$O(9C'(C+C')H'W')$	$9C'(C+C')$	both convolutions
elastic distortion	$O(HW\sigma)$	0	—

Where the cost goes. Because each weight is reused at every position (Worked example 6), a convnet performs many operations per parameter: it is *compute-bound*, which is why GPUs mattered so much in 2012 (Section 7).

Summary Convolutional networks are MLPs with two constraints on the weights: each unit sees only a local patch of the input, and all units of a feature map share the same weights. A layer computes cross-correlations of all input channels with a $C_{\text{in}}\times k\times k$ filter per output channel and has $C_{\text{out}}(C_{\text{in}}k^2+1)$ parameters, independently of the image size. Convolution commutes with translations (equivariance); pooling summarises neighbourhoods and adds approximate local invariance and subsampling. Other invariances are taught by data augmentation, including elastic distortions. Backpropagation through a convolution sums the gradients of a shared weight over positions and sends the output gradient back by the transposed operation, a full convolution. Stacks of small filters, 1×1 channel mixing and residual blocks took convnets from AlexNet to networks hundreds of layers deep. The output size along an axis is $\lfloor (i+2p-k)/s \rfloor + 1$; dilation enlarges the effective kernel to $k+(k-1)(d-1)$ and gives exponentially large receptive fields at linear cost; transposed convolutions and resize-convolutions enlarge feature maps for segmentation and generation, and U-Nets combine both directions with skip connections.

Questions and exercises

Questions are sorted by kind. Parts A and B are of the kind used in the midterm examination: they test understanding, interpretation and short calculations (at most simple derivatives), not long proofs. Parts C and D go further and are recommended for practice and for the project.

A. Conceptual questions

A1. Name the three ideas that distinguish a convolutional layer from a fully connected one. For each, state the property of images it exploits and what it saves (parameters, computation, or data).

A2. Explain why a convolutional layer can be seen as a fully connected layer with constraints on its weight matrix. Which entries are forced to be zero, and which are forced to be equal?

A3. A convolutional layer with 3×3 kernels is applied to a 64×64 image and then to a 512×512 image. Does the number of parameters change? Does the computation change? Explain.

A4. Define translation *equivariance* and translation *invariance*. Which of the two does a convolutional layer have, which does max pooling approximately add, and which does a classifier ultimately need?

A5. What is the difference between convolution and cross-correlation? Why does it not matter for a layer whose kernel is learned, and when does it matter?

A6. A filter of the first layer of a network for colour images has size $3 \times 5 \times 5$. Explain what the three numbers mean and why the first one must equal the number of input channels.

A7. Explain why max pooling gives only approximate invariance to translations. Describe a shift by one pixel that changes the output of 2×2 max pooling, and one that does not.

A8. In a LeNet-style network most parameters are in the fully connected layers and most computation in the convolutional layers. Explain why.

A9. How does the receptive field of a unit grow with depth in a stack of stride-1 3×3 convolutions? What do pooling and dilation change?

A10. Why does a convnet trained without augmentation not automatically recognise rotated digits? Give one transformation that is a sensible augmentation for photographs of animals but not for handwritten digits, and explain.

A11. In an elastic distortion, what do the width σ and the intensity α control? What goes wrong for very small σ?

A12. Name the four ingredients that AlexNet combined, and say for each which section or note of this course explains it.

A13. Why do two stacked 3×3 convolutions often replace one 5×5 convolution? Give two reasons.

A14. What problem did residual connections solve in very deep convnets, and why was it an optimisation problem rather than overfitting? What must the shortcut do when a block changes the number of channels or uses stride 2?

A15. Explain the role of a 1×1 convolution. Why is it called a fully connected layer applied at every pixel?

A16. What does "same" padding achieve, and why is it awkward for even kernel sizes?

A17. Explain how global average pooling allows a network to accept images of different sizes.

A18. Jets in a calorimeter image can be handled by augmentation (random rotations) or by standardisation (rotating every jet into a fixed orientation). Give one advantage of each, and say when only augmentation is possible.

A19. Why is a convolution of a 224×224 image best stored as a kernel and not as the matrix C of Section 8.1? What does the matrix view nonetheless explain about the backward pass?

A20. What is a causal convolution, and why does audio generation need it? Why is generation with WaveNet much slower than training?

A21. Why is a transposed convolution not the inverse of a convolution? Explain where checkerboard artefacts come from and two ways to avoid them.

A22. In a U-Net, what do the skip connections carry, and why can the decoder not recover this information on its own?

A23. The patch embedding of a vision transformer is a convolution. With which kernel size and stride? Which prior of convnets does the rest of a transformer give up, and what is the price in terms of data?

B. Short calculations

B1. Compute the cross-correlation and the convolution (valid, stride 1) of $x = (1, 3, 0, -2, 4)$ with $w = (2, -1)$. Then compute the cross-correlation with $w = (1, 1, 1)/3$ and describe what this kernel does.

B2. Use the image x and the kernel w of Worked example 3. (a) With zero padding $p = 1$ and stride 1, compute the first row of the 4×4 output. (b) With $p = 1$ and stride 2, how large is the output? Compute its top-left entry. (c) Apply ReLU to the 2×2 output of Worked example 3 with bias 0 and then 2×2 max pooling.

B3. Count the parameters and the forward multiply–adds of a convolutional layer with $C_{\text{in}} = 32$, $C_{\text{out}} = 64$, $k = 3$, stride 1 and same padding on a 32×32 input (include the biases). Repeat for a depthwise separable layer (one 3×3 filter per input channel, then a 1×1 convolution from 32 to 64 channels), and give the ratio.

B4. Compute the output size for $(i, k, s, p) = (32, 5, 1, 0)$, $(32, 5, 1, 2)$, $(32, 3, 2, 1)$, $(33, 3, 2, 1)$, and for a dilated convolution with $i = 32$, $k = 3$, $d = 4$, $s = 1$, $p = 0$. Which padding makes the dilated convolution "same"?

B5. Max-pool the map

$$h = \begin{pmatrix} 1 & 4 & 0 & 2 \\ 3 & 0 & 1 & 5 \\ 2 & 1 & 6 & 0 \\ 0 & 1 & 3 & 1 \end{pmatrix}$$

in non-overlapping 2×2 blocks. If the gradient of the loss with respect to the pooled map is

$$\begin{pmatrix} 1 & -1 \\ 2 & 0.5 \end{pmatrix},$$

write down the gradient with respect to the 4×4 map. Repeat for average pooling.

B6. A one-dimensional layer computes $y_j = \sum_{m=0}^{1} w_m x_{j+m}$ with $w = (1, 2)$ on $x = (0, 1, -1)$. With the loss $\ell = \frac{1}{2}\sum_j y_j^2$, compute y, $\partial \ell / \partial w_0$, $\partial \ell / \partial w_1$ and $\partial \ell / \partial x_q$ for $q = 0, 1, 2$ using (8) and (9). Check one of the weight gradients by expanding ℓ as a function of w_0.

B7. Compute the receptive field, in input pixels, of a unit after the sequence conv 3×3, conv 3×3, max pool 2×2, conv 3×3, conv 3×3, max pool 2×2 (all convolutions stride 1). How many layers of causal dilated convolutions with $k = 2$ and $d = 1, 2, 4, \ldots$ are needed for a receptive field of at least one second of audio at 16 kHz?

B8. Compute the transposed convolution of $\mathbf{y}' = (1, -1, 2)$ with the kernel $(1, 1)$ and stride 2, and with the kernel $(1, 1, 1)$ and stride 2 (no padding). Give the output lengths from (13). For a constant input, which of the two kernels produces an uneven pattern?

C. Derivations

C1. Let A be a linear map on infinite sequences $(x_j)_{j \in \mathbb{Z}}$ that commutes with all shifts, $AT_s = T_s A$. Let $\mathbf{e}$ be the sequence with $e_0 = 1$ and zeros elsewhere, and $g = A\mathbf{e}$. Show that $(Ax)_j = \sum_q g_{j-q}\, x_q$ for every finitely supported x, so that A is a convolution with kernel g. Then show that the complex exponentials $x_j = e^{\mathrm{i}\omega j}$ (with $\mathrm{i} = \sqrt{-1}$) are eigenvectors of every convolution and give the eigenvalue.

C2. Derive (8) and (9) for a one-dimensional convolution with stride s and padding p, stating which input positions receive non-zero gradients when s does not divide $i + 2p - k$. Write the input gradient as a transposed convolution of the output gradient.

C3. Derive the receptive-field recursion (7). Deduce that L stride-1 layers with kernel size k and dilations d_l have receptive field $1 + (k - 1)\sum_l d_l$, and that the choice $d_l = k^{l-1}$ gives k^L.

C4. Show that the transposed convolution with stride s and kernel w of length k, applied to a constant input, gives a constant output away from the borders if and only if s divides k (for a kernel of all ones). What is the output pattern for general k and s?

D. Programming

D1. Implement Algorithms 1 and 3 in numpy for one example with several channels, stride s and padding p. Check the gradients as in Note 12, Section 4, on random inputs, and compare the forward pass with `torch.nn.functional.conv2d`. Then implement Algorithm 7 and compare its speed with your loop version for a 64-channel 56×56 input.

D2. In PyTorch, train the LeNet-style network of Worked example 6 on MNIST (or on synthetic 28×28 images of shapes) with and without elastic distortions (Algorithm 4) and random shifts of up to two pixels. Compare test errors and learning curves for training sets of 1000 and 10 000 images. Then evaluate both models on test images rotated by 10°, 20° and 30°.

D3. Generate Ising configurations on a 16×16 periodic lattice by Metropolis sampling at temperatures between 1.5 and 3.5 (in units of J/k_B) and train a small convnet with circular padding and global average pooling to classify $T < T_c$ against $T > T_c$, with $T_c \approx 2.269$. Plot the predicted probability of the ordered phase against T. Then apply the same trained network, unchanged, to 32×32 lattices and compare.

Further reading. Goodfellow, Bengio and Courville [12], chapter 9; Prince [29], chapters 10 and 11; Bishop [1], section 5.5, for invariances and convolutional networks; Murphy [27], chapter 14; Dumoulin and Visin [9] for the arithmetic of Section 8, with many diagrams.

References

[1] C. M. Bishop, *Pattern Recognition and Machine Learning*, Springer (2006).

[2] J. Carrasquilla and R. G. Melko, Machine learning phases of matter, *Nature Phys.* 13, 431–434 (2017).

[3] T. Chen, S. Kornblith, M. Norouzi and G. Hinton, A simple framework for contrastive learning of visual representations, *ICML* (2020).

[4] J. Cogan, M. Kagan, E. Strauss and A. Schwartzman, Jet-images: computer vision inspired techniques for jet tagging, *JHEP* 02, 118 (2015).

[5] T. S. Cohen and M. Welling, Group equivariant convolutional networks, *ICML* (2016).

[6] J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li and L. Fei-Fei, ImageNet: a large-scale hierarchical image database, *CVPR* (2009).

[7] L. de Oliveira, M. Kagan, L. Mackey, B. Nachman and A. Schwartzman, Jet-images — deep learning edition, *JHEP* 07, 069 (2016).

[8] A. Dosovitskiy *et al.*, An image is worth 16×16 words: transformers for image recognition at scale, *ICLR* (2021).

[9] V. Dumoulin and F. Visin, A guide to convolution arithmetic for deep learning, arXiv:1603.07285 (2016).

[10] K. Fukushima, Neocognitron: a self-organizing neural network model for a mechanism of pattern recognition unaffected by shift in position, *Biol. Cybern.* 36, 193–202 (1980).

[11] D. George and E. A. Huerta, Deep neural networks to enable real-time multimessenger astrophysics, *Phys. Rev. D* 97, 044039 (2018).

[12] I. Goodfellow, Y. Bengio and A. Courville, *Deep Learning*, MIT Press (2016).

[13] K. He, X. Zhang, S. Ren and J. Sun, Deep residual learning for image recognition, *CVPR* (2016).

[14] J. Ho, A. Jain and P. Abbeel, Denoising diffusion probabilistic models, *NeurIPS* (2020).

[15] A. G. Howard *et al.*, MobileNets: efficient convolutional neural networks for mobile vision applications, arXiv:1704.04861 (2017).

[16] I. Goodfellow, D. Warde-Farley, M. Mirza, A. Courville and Y. Bengio, Maxout networks, *ICML* (2013).

[17] J. Hu, L. Shen and G. Sun, Squeeze-and-excitation networks, *CVPR* (2018).

[18] D. H. Hubel and T. N. Wiesel, Receptive fields, binocular interaction and functional architecture in the cat's visual cortex, *J. Physiol.* 160, 106–154 (1962).

[19] N. Kalchbrenner, L. Espeholt, K. Simonyan, A. van den Oord, A. Graves and K. Kavukcuoglu, Neural machine translation in linear time, arXiv:1610.10099 (2016).

[20] A. Krizhevsky, I. Sutskever and G. E. Hinton, ImageNet classification with deep convolutional neural networks, *NeurIPS* (2012).

[21] Y. LeCun, B. Boser, J. S. Denker, D. Henderson, R. E. Howard, W. Hubbard and L. D. Jackel, Backpropagation applied to handwritten zip code recognition, *Neural Comput.* 1, 541–551 (1989).

[22] Y. LeCun, L. Bottou, Y. Bengio and P. Haffner, Gradient-based learning applied to document recognition, *Proc. IEEE* 86, 2278–2324 (1998).

[23] M. Lin, Q. Chen and S. Yan, Network in network, *ICLR* (2014).

[24] Z. Liu, H. Mao, C.-Y. Wu, C. Feichtenhofer, T. Darrell and S. Xie, A ConvNet for the 2020s, *CVPR* (2022).

[25] J. Long, E. Shelhamer and T. Darrell, Fully convolutional networks for semantic segmentation, *CVPR* (2015).

[26] P. Mehta and D. J. Schwab, An exact mapping between the variational renormalization group and deep learning, arXiv:1410.3831 (2014).

[27] K. P. Murphy, *Probabilistic Machine Learning: An Introduction*, MIT Press (2022).

[28] A. Odena, V. Dumoulin and C. Olah, Deconvolution and checkerboard artifacts, *Distill* (2016).

[29] S. J. D. Prince, *Understanding Deep Learning*, MIT Press (2023).

[30] R. Rombach, A. Blattmann, D. Lorenz, P. Esser and B. Ommer, High-resolution image synthesis with latent diffusion models, *CVPR* (2022).

[31] O. Ronneberger, P. Fischer and T. Brox, U-Net: convolutional networks for biomedical image segmentation, *MICCAI* (2015).

[32] O. Russakovsky *et al.*, ImageNet large scale visual recognition challenge, *Int. J. Comput. Vis.* 115, 211–252 (2015).

[33] P. Y. Simard, D. Steinkraus and J. C. Platt, Best practices for convolutional neural networks applied to visual document analysis, *Proc. ICDAR* (2003).

[34] K. Simonyan and A. Zisserman, Very deep convolutional networks for large-scale image recognition, *ICLR* (2015).

[35] C. Szegedy *et al.*, Going deeper with convolutions, *CVPR* (2015).

[36] N. Thuerey, K. Weißenow, L. Prantl and X. Hu, Deep learning methods for Reynolds-averaged Navier–Stokes simulations of airfoil flows, *AIAA J.* 58, 25–36 (2020).

[37] A. van den Oord *et al.*, WaveNet: a generative model for raw audio, arXiv:1609.03499 (2016).

[38] F. Yu and V. Koltun, Multi-scale context aggregation by dilated convolutions, *ICLR* (2016).

[39] M. D. Zeiler and R. Fergus, Visualizing and understanding convolutional networks, *ECCV* (2014).

[40] H. Zhang, M. Cisse, Y. N. Dauphin and D. Lopez-Paz, mixup: beyond empirical risk minimization, *ICLR* (2018).

The selection and order of topics in these notes follow the course IFT 6390 *Fondements de l'apprentissage machine* at Université de Montréal (taught by Ioannis Mitliagkas, slides by Pascal Vincent). The text, examples, figures and exercises of this note are original. Figures are computed from synthetic data by `figures.py`, published with the sources of these notes.

The IFT 6390 slides on convolutional networks are based on material by Hugo Larochelle.